Shielding Against Phishing Attacks

A Major Project Report Submitted
In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
In
Computer Science and Engineering

By

Adelli Manasa	**2011CS010012**
Darapuneni vivek	**2011CS010076**
Dutta Goutham Satya Sai Kiran	**2011CS010375**
Vuppula Karthik Reddy	**2011CS010037**

Under the esteemed guidance of

Mr.M.RAKESH
ASSISTANT PROFESSOR

Department of Computer Science and Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana - 500100

2024

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that this is the bonafide record of the project entitled **"Shielding Against Phishing Attacks"**, submitted by **Adelli Manasa (2011CS010012), Darapuneni Vivek (2011CS010076), Dutta Goutham Satya Sai Kiran (2011CS010375), Vuppula Karthik Reddy (2011CS010037)**, towards the partial fulfillment for the award of Bachelor's Degree in Computer Science and Engineering, Department of CSE, Mall Reddy University, Hyderabad, during the year 2023-2024.. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

Internal Guide	**Head of the Department**
Mr.M.Rakesh	**Dr.Meeravali Shaik**
Assistant Professor	**M.Tech., Ph.D.**

External Examiner

DECLARATION

We hereby declare that the project report entitled **"Shielding Against Phishing Attacks"** has been carried out by us and this work has been submitted to the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad in partial fulfillment of the requirements for the award of degree of Bachelor of Technology. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place: Hyderabad
Date: 03-05-2024

Adelli Manasa	**2011CS010012**	**(A.Manasa)**
Darapuneni Vivek	**2011CS010076**	**(D.Vivek)**
Dutta Goutham Satya Sai Kiran	**2011CS010375**	**(D.Goutham)**
Vuppula Karthik Reddy	**2011CS010037**	**(V.Karthik)**

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, We would like to extend our gratitude to **Dr. V. S. K Reddy**, Vice-Chancellor, for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express my deepest appreciation to our project guide **Mr.M.Rakesh**, Assistant Professor, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We are also grateful **to Dr.Meeravali Shaik**, Head of the Department of CSE, for providing us with the necessary resources and facilities to carry out this project.

We would like to thank **Dr. Kasa Ravindra**, Dean, School of Engineering, for his encouragement and support throughout my academic pursuit.

My heartfelt thanks also go to **Dr. Harikrishna Kamatham**, Associate Dean School of Engineering for his guidance and encouragement.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

Adelli Manasa	2011CS010012
Darapuneni Vivek	2011CS010076
Dutta Goutham Satya Sai Kiran	2011CS010375
Vuppula Karthik Reddy	2011CS010037

ABSTRACT

XSS Attacks, SQL Injection, and Online Phishing are three of the most common Internet crime techniques. Checking URLs against blacklists of known phishing websites, which are generally built based on manual verification, is a frequent counter-measure for Phishing Website Detection and is inefficient. A typical Internet user will not be able to distinguish between XSS Attack and SQL Injection Attack.

As a result, as the Internet's scale expands, technology and its components advance to give timely protection to end users. We present in this project an effective and versatile malicious URL detection system with a rich collection of features that reflect many aspects of phishing webpages and their hosting platforms, as well as the detection of other vulnerabilities such as XSS Attack and SQL Injection using web scraping and a dummy code injection mechanism are used in an injection attack.

Our project consists of three modules, for each vulnerability we have created a module. We used Jupyter Notebook, Matplotlib, and NumPy packages for building the random forest model and incorporated this trained model in the GUI which is created in Jupyter Notebook. It allows users to check any link just by copying it into the GUI and gives instant results. Random forest algorithms are used to detect phishing. Injecting SQL payloads and malicious scripts into the input fields of the given URL is the test of and detect the vulnerability of SQL Injections and Reflected XSS. The user just needs to run the GUI module and should have to provide the URL as an input and in the backend, all these modules run and provide the user with an output in the GUI.

Our system has high detection power and low error rates, thanks to the Random Forest Algorithm. This is the first study that we are aware of that uses effective feature collecting to conduct a Vulnerability Detection process of large-scale websites/URLs. The results of the experiments show that our system is capable of efficiently finding dangerous websites/URLs by the end user.

TABLE OF CONTENTS

S.NO	TITLE	PG.NO
1	INTRODUCTION	10-15
	1.1 INTRODUCTION	10-13
	1.2 PROBLEM STATEMENT	13
	1.3 SCOPE	13
	1.4 OBJECTIVE	13-14
	1.5 IMPORTANT TERMS AND CONCEPTS	14-15
2	LITERATURE SURVEY	16-23
	2.1 FEASIBILITY STUDY	16-17
	2.2 LITERATURE SURVEY	18-23
3	SOFTWARE REQUIREMENT ANALYSIS	24-27
	3.1 SOFTWARE REQUIREMENTS	24-27
4	SOFTWARE DESIGN	28-33
	4.1 USE CASE DIAGRAM	28-29
	4.2 CLASS DIAGRAM	30
	4.3 SEQUENCE DIAGRAM	31
	4.4 ACTIVITY DIAGRAM	32
	4.5 FLOWCHART	33
5	PROPOSED SYSTEM	34-42
	5.1 DETECTION OF SQL INJECTION	35-37
	5.2 DETECTION OF XSS	37-39
	5.3 DETECTION OF PHISHING	39-40
	5.4 BUILDING GRAPHICAL USER INTERFACE	41
6	IMPLEMENTATION	43-53
7	TESTING	54-56
8	OUTPUT SCREENS	57-59
9	CONCLUSION	60
10	FUTURE WORK	61
11	REFERENCES	62

LIST OF FIGURES

FIGURE.NO	NAME	PG.NO
1.1	PIE CHART OF PERCENTAGE OF DIFFERENT CYBER-ATTACKS	11
1.2	HOW RANDOM FOREST WORKS	15
4.1	USE CASE DIAGRAM	29
4.2	CLASS DIAGRAM	30
4.3	SEQUENCE DIAGRAM	31
4.4	ACTIVITY DIAGRAM	32
4.5	FLOW CHART OF THE PROJECT	33
5.1	FLOW DIAGRAM OF SQL INJECTION	36
5.2	LIST OF ERRORS THAT ARE THROWN BY SQL VULNERABLE WEBSITES	36
5.3	FLOW CHART OF XSS VULNERABILITY	38
5.4	CODE OF INJECTING JAVASCRIPT CODE INTO FORMS	38
5.5	STRUCTURE OF GRAPHICAL USER INTERFACE	41
5.6	TIER ARCHITECTURE	42
5.7	WORKFLOW	42
6.1	PHISHING MODULE - 1	43
6.2	PHISHING MODULE - 2	44
6.3	GUI MODULE - 1	45

6.4	**GUI MODULE - 2**	**46**
6.5	**GUI MODULE - 3**	**47**
6.6	**SQL INJECTION MODULE - 1**	**48**
6.7	**SQL INJECTION MODULE - 2**	**49**
6.8	**SQL INJECTION MODULE - 3**	**50**
6.9	**XSS MODULE – 1**	**51**
6.10	**XSS MODULE – 2**	**52**
6.11	**XSS MODULE – 3**	**53**
7.1	**TESTCASE – 1**	**54**
7.2	**TESTCASE – 2**	**55**
7.3	**TESTCASE – 3**	**56**

LIST OF OUTPUTS

FIGURE.NO	NAME	PG.NO
8.1	OUTPUT - 1	57
8.2	OUTPUT – 2	58
8.3	OUTPUT - 3	59

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Over the past years, we have seen enormous growth in internet connectivity and usage. The internet is everywhere, and it has become the main aspect for all types of users ranging from building communication between two common people to sharing highly confidential information between two countries. The Internet has become just like the five elements which are essential for human survival. Even in the financial sectors internet plays a vital role. The best way to maximize the online presence on the internet is through websites. The website had become the main expansion marketing strategy for business. Even for a small-scale business, there is a website. The reason for this is that building a basic static website without any advanced security features takes less time and cost.

Even though the internet has so many upsides there is always a risk of privacy and data safety. These days the internet is embedded into many devices increasing the risk of data privacy. Since Internet has been updating for the past decade and developers always strive to make their websites look visually more appealing rather than improving their security standards in the fear of poor performance of websites. This leads to unethical coding practices and hackers always try to take advantage of these websites and capitalize on them through stealing vital information.

The point present in a website where hackers enter to attack and break the website is generally called a vulnerability. As the number of web apps grows, vulnerabilities have become a hot topic in many web-application development and security discussion groups. Web applications normally let the immediate and repeated usage of sensitive consumer data be captured, processed, stored, and transmitted. As a result, web applications have become key targets for attackers that take advantage of web developers' poor coding skills and application code gaps., inadequate authorization of user’s inputs, or failure by software development developers to comply with security requirements. These flaws could be found on either the server or the client side. Cross-Site Scripting, Phishing, SQL injection, Cross-Site Request Forgery, Broken Authentication, and other vulnerabilities

are among them. This project deals with the client- side vulnerabilities (Cross-Site Scripting), server-side (SQL injection), social Engineering (phishing).

Fig.1.1: Pie chart of percentage of different cyber-attacks

Cross-site scripting is a vulnerability in web security that allows an attacker to jeopardize user engagement with a vulnerable application. It usually allows an attacker to disguise as a victim, perform any actions that a user can carry out, and view any data that is available to the user. It also allows an attacker to bypass the same strategy of roots, which is intended to separate different websites one from the other. If the user has unrestricted permissions, the attacker can acquire full control over all the features and the information of the web application. It works by handling vulnerable websites so that JavaScript outputs malicious to users.

The attacker can threaten their ability to interact with the application when a mischievous code runs inside the client's webpage. In a webpage, if the user present on the client-side provides a simple JavaScript code as an input on any one of the forms present in the webpage. If the website executes the JavaScript code as a server input instead of client input (as a String) then that webpage has a cross-site scripting vulnerability. In any Cross-site vulnerable websites, the hacker incorporates mischievous JavaScript code which will leak the confidential data of the organization. Cross-site scripting vulnerabilities can be

divided into three categories. Reflected XSS, Stored XSS, and DOM-based XSS are the three types.

This project mainly concentrates on Reflected and DOM based cross-site vulnerabilities. A reflected XSS is also called a non- persistent XSS attack. It jumps off another website using a malicious script in the browser of a victim. The query is usually passed through the URL. The Document Object Model Based XSS simply defines that vulnerability displays in the DOM instead of part of HTML.

SQL injection is a type of website vulnerability allowing a hacker to interfere with queries made in its database by an application. It usually allows a hacker to view data which they usually cannot recover. This may include data from other users, or other data accessible to the application itself. In several cases, this can be modified or deleted by an attacker which causes persistent changes in the content or behavior of the application. In certain scenarios, the hacker tries to break the underlying base server and perform a DOS attack. If a hacker tries to successfully perform an attack by SQL injection may lead to illegitimate access to confidential data like passwords, credit card details, or personal user data. Many well-known data breaches in subsequent years have resulted in reputational impairment and regulatory penalties caused by SQL injection attacks. In rare situations, an attacker can gain a persistent backdoor into an organization's systems, resulting in a long-term breach that goes unreported for longer.

If the website executes the query code or special characters like a quote (') as a server Input instead of client input (as a String) then that particular webpage has a SQL injection vulnerability. In any SQL injection vulnerable websites, the hacker incorporates mischievous query code which will leak the confidential data and gains the credentials of an administrator in an organization.

Phishing is a type of social engineering attack that is commonly used to obtain personal information from users, such as bank account information and authentication tokens. It occurs when an attacker conceals an illegitimate website into a trustworthy entity. The recipient is tried to click on a malicious connection that could lead to malware being installed, the system freezing as part of a ransomware attack, or sensitive information being disclosed. An assault can be devastating. This includes unauthorized shopping, fund stealing, or identification of theft for individuals.

In addition, phishing has been used as a part of a major attack, as in an “advanced persistent threat (APT)”, to gather a foothold in private or government sectors. Therefore, in latter circumstances, employees will be compromised to sneak past security limits, install malicious objects in closed environments or access privileged confidential information. A company that gone through such attacks suffers from decreasing market share, credibility, and consumer confidence, which typically affects severe revenue damage. Depending on the scope, a phishing attempt may turn into a safety incident where a company has a hard time recuperating. Examples of phishing include hackers create a replica of a legitimate webpage and send it to various clients hoping the client might enter confidential information.

Since phishing is mostly dependent on the user’s way of thinking and awareness of the concepts of cybersecurity. As a result, a variety of machine learning techniques are utilized to determine whether a webpage is phishing or not. This project uses the most popular random forest algorithm which provides high accuracy with less amount of time.

1.2 PROBLEM STATEMENT

To identify the three most common vulnerabilities namely phishing (social engineering) websites using machine learning algorithms (Random Forest) , Payload SQL injection vulnerability (server side) in web pages which contain canonical URLs and Reflected XSS vulnerability (client side) in web page which contain the ability to interact with user. To develop a Graphical User Interface that can help a general user to identify the above mentioned vulnerabilities in a website

1.3 SCOPE

The scope of this application is to detect the three vulnerabilities named SQL Injection , Cross- site Scripting and Phishing in a website

1.4 OBJECTIVE

The main objective of this project is to develop a platform-independent, user-friendly, GUI based application that can detect social engineering, payload SQL injection, and

reflect cross- site vulnerabilities using different algorithms. This application will be visually aesthetic. The result will be a simple GUI, which will be able to detect vulnerabilities with high precision in a short amount of time. The user does not need to know the packages or technologies that were used to build this application beforehand. The customer just needs to know what type of vulnerabilities this application detects. The user provides the GUI with the URL, which indicates that the application detects vulnerabilities to the provided URL of a website.

1.5 IMPORTANT TERMS AND CONCEPTS

1.5.1 Random forest

Random forest is a part of machine learning algorithm to solve classification and regression problems. It makes use of ensemble learning, which is a algorithm to solve complicated or tough problems by merging multiple classifiers.

Many decision trees combines to form random forest algorithm. Bagging or bootstrap aggregation are used to train the 'forest'(dense) formed by the random forest method. Bagging is a meta-algorithm that improves the accuracy of machine learning algorithms by grouping or merging them together.
The (random forest) algorithm displays the outcome based on decision tree predictions. It forecasts through the output average of various trees. Precision of the result improves as the number of trees (decision trees) grows.

1.5.2 How random forest algorithm works

Gaining An understanding of Decision Trees

A Random Forest Algorithm components are decision trees. A decision tree is a decision making tool just like a tree-like structure. A basic concept of decision trees will be a part of our understanding in random forest algorithms.

There are three parts in a decision tree: decision nodes, leaf nodes, and a root node.

A decision tree method separates a training dataset into branches, each of which is again divided into branches. This pattern repeats till a leaf node is reached. There is no other way to divide the leaf node any farther.

The attributes are used to forecast the outcome which are represented by the nodes in the decision tree. The leaves are connected to the decision nodes in the decision tree. The three types of nodes in a decision tree are shown in the diagram below

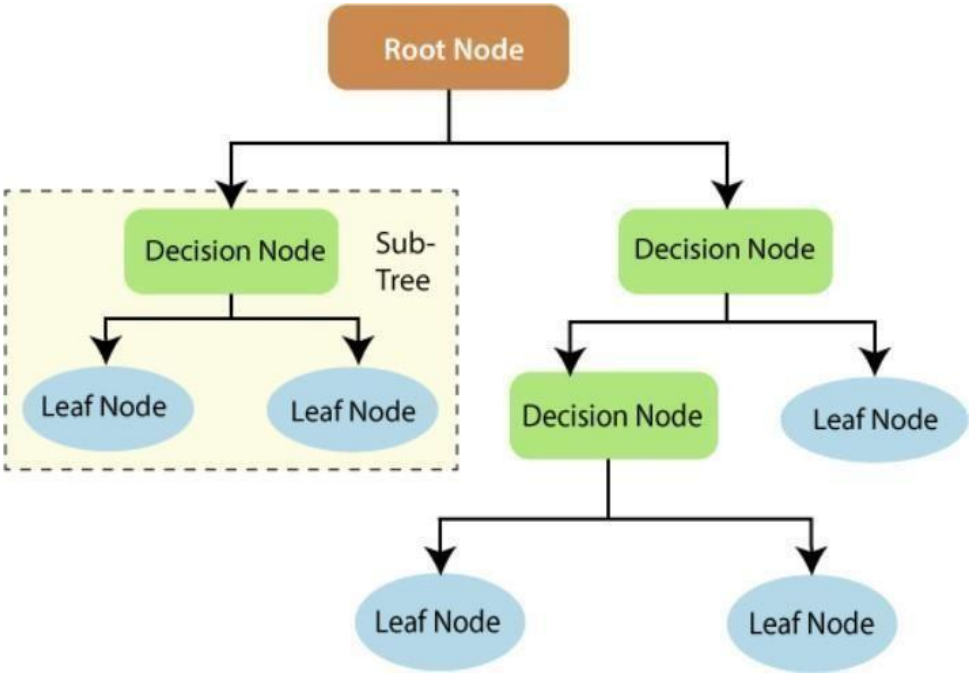

Fig 1.2: How random forest work

CHAPTER 2

LITERATURE STUDY

2.1 FEASIBILITY STUDY

A project's importance impacts how successfully it can be completed. A feasibility study is carried out to see if the chosen solution is practical and meets the project criteria. The feasibility study considers data such resources, software cost estimates, business advantages, and software maintenance expenses. The feasibility study's purpose is to develop the foundation for publishing software that is simple to use, adaptable, resilient, and adheres to established standards.

2.1.1 Organizational Feasibility

The effectiveness of the proposed information system in meeting the organization's purpose and strategic information system strategy is one of the most critical aspects in organizational feasibility. It could include information on the founders, their education, and the abilities they need to establish and maintain a company. It also impacts whether users are able to start the system and receive sufficient user help to perform their tasks effectively. Projects that do not directly serve the company's strategic goal, for example, are frequently not supported.

2.1.2 Economical Feasibility

Economic feasibility is a type of cost-benefit analysis used to determine whether or not a project can be carried out. This phrase refers to the examination and study of a project's ability to assist decision-making by objectively and logically identifying its strengths, flaws, opportunities, and dangers, as well as the resources needed to complete it and an estimate of its chances of success. The Market analysis, economic analysis, technical analysis, and strategic analysis were all included.

2.1.3 Technical Feasibility

Technical feasibility evaluates the expert system's technical complexity and typically includes determining whether the expert system can be employed with cutting-edge procedures and equipment. For expert systems, determining the shell in which the system will be built is critical since it influences technical feasibility. The development environment can have a significant impact on the quality of an expert system, making the

shell a critical component of the system's success. Although the optimal attributes of an expert system shell may vary depending on the task and domain requirements, the shell must be sufficiently adaptive to allow expert reasoning to be successfully integrated into the system. It also needs to be straightforward to integrate with current computer-based systems.

2.1.4 Behavioural Feasibility

Change has been facilitated by computers in the past, but humans are intrinsically resistant to change. An estimate of the possibility of user personnel reacting unfavorably to the installation of an automated system should be made. Employee job status, relocations, retraining, and turnover have all been known to be influenced by IT installations. As a result, it's understandable that putting in place a candidate system necessitates a concerted effort to explain, persuade, and educate employees about new business processes.

2.2 LITERATURE SURVEY

Number	Authors	Paper Title and Year of Publication	Methodology and Drawbacks
1	Waleed Ali and Sharaf Malebary, (Member, IEEE)	Particle Swarm Optimization-Based Feature Weighting for Improving Intelligent Phishing Website Detection (19 June 2020)	**Methodology**: utilizing particle swarm optimization (PSO) to weight various website features effectively. **Drawbacks:** The analysis and weighting of features using the ML algorithm takes longer.
2	Yazan Ahmad Alsariera, Victor Elijah Adeyemo, Balogun and Ammar Kareem Alazzawi	AI Meta-Learners and Extra-Trees Algorithm for the Detection of Phishing Websites (3-Aug-2020)	**Methodology**: 4 AI based meta learner models were developed using extra tree base classifier for detecting phishing websites. **Drawbacks**: To build an interpretable model, hybridization can be done using alternative decision tree techniques. .
			Methodology: To detect classic XSS flaws, fuzzy logic was employed.

3	Bakare K. Ayeni, Junaid u B. Sahalu, and Kolawole R. Adeyanju	Detecting XSS in Web Applications Using Fuzzy Inference System (1-Jan-2018)	**Drawbacks**: We may create a model to detect various types of code and server-side injection flaws, such as SQL and cross-site request forgery.
4	Xin Xie, Chunhui Ren, Yusheng Fu, Jie Xu, Jinhong Guo	SQL Injection Detection for Web Applications Based on Elastic-Pooling CNN (21 Oct 2019)	**Methodology**: Elastic pooling CNN approach is used for detecting the vulnerabilities. **Drawbacks**: It cannot identify new attacks.
5	Raima Banerjee, aitra baksi, nidhi singh and soham kanti bishnu	Detection of XSS in web application using ml classifiers. (30-Nov-2020)	**Methodology** : Building classifiers using machine learning algorithms to detect the severity of the XSS attack. **Drawbacks**: There is a 0.34 false positive rate, which can be addressed.
6	Oluwakemi christiana , Abdullah i abubakar	A novel technique to prevent SQL injection and XSS attacks using KMP string matching algorithm (2020)	**Methodology**: Threats are detected using KMP string matching algorithm **Drawbacks**: Un-stored patterns cannot be recognized

7	Arun Prasath , Akshay Mathur , javaid	Browser extension for detecting XSS attack in HTML5 based websites (2018)	**Methodology**: A plugin that alerts whenever a malicious code is detected **Drawbacks** : It can only be used in html5 based websites.
8	Mahdieh Zabihimayvan and Derek Doran	Fuzzy Rough Set Feature Selection to Enhance Phishing Attack Detection (10 October 2019)	**Methodology** : Fuzzy rough set theory is used to select effective features and fed to classifiers. **Drawbacks**: Accuracy can be improved.
9	Chen Ping	A second-order SQL injection detection method (08 Feb 2018)	**Methodology**: It is based on instruction set randomization. **Drawbacks**: Efficiency can be improved.
10	Mohith Gowda HR, Aditya MV, Ganesh Prasad and Vinay S	Development of anti-phishing browser based on random forest and rule of extraction Framework (14 oct 2020)	**Methodology**: To detect the fault, properties are retrieved from the URL and supplied to a random forest classifier. **Drawbacks**: It employs only one trained model and necessitates the use of unsupervised learning.

Table 2.1: The table gives brief description about few papers

In this paper[1] Detection of XSS attacks in a web application using Machine Learning Classifiers was done. Here various classifiers were taken into consideration and trained using a vast amount of data from various datasets. Based on the prediction the websites will be classified as either vulnerable or not vulnerable .The drawback of this particular approach is that the efficiency can be improved using hybrid models or combinations of classifiers.

In this paper[2] Detection of SQL Injection attacks using a machine learning approach was done. Here an ML based heuristic algorithm was proposed in order to prevent the SQL injection attack. Here they have tried to use select number of SQL tags or statements in order to test the classifier. But they may lead to an inefficient classifier model if the attacker tries to use combination of SQL statements.

In this paper[3] It was decided to use a fuzzy rough set feature selection to improve Phishing attack detection. The features retrieved from the example websites are totally reliant on this technique. Random forest Classifier model was used. Features were extracted from 14,000 websites. Higher accuracy can be obtained if hybrid models or combination of models are used in the application.

In this paper[4] SQL Injection detection for web applications on Elastic-pooling CNN was taken. Based on the irregularity in matching characteristics, it can identify new attacks and it is harder to configure. The mechanism uses the elastic pooling method of CNN in order to identify the patterns of the input and then if the pattern matches with the previously identified malicious pattern the website is said to be affected with the SQL injection attack. In this way this approach works. The proposed method is only built to identify SQL injection attack but we have focused on similar attacks like XSS and Phishing.

The paper[5] Machine learning based phishing detection from URLs was done. The URL is taken as the input and is splitted into various strings and they are passed through a machine learning classifier which will predict the URL as a phishing website or not. In this way the following approach works. The usage of models as well as the diverse number of datasets can be used in order to improve the efficiency and accuracy.

In this paper[6] A google chromium browser extension for detecting XSS attack in HTML5 based websites is built. The plugin acts as an alert mechanism for the web browser and detects if there is any vulnerability in the website. The input fields are monitored by the plugin and if any of the HTML tags are identified then it alerts the website of the XSS attack and prevents the hacker from performing the attack. The main disadvantage of this project is that it is only relevant to HTML-based websites and that the user must install the plugin in order for it to work.

In this paper[7] A fuzzy inference technique was used to detect XSS in web applications. It describes a smart method for identifying cross-site scripting (XSS) assaults. To detect classic XSS flaws, this solution was based on fuzzy logic. The users can utilize this technique as a decision-making tool. This technique can only detect classic XSS problems, according to the report. This can be considered as a drawback since hackers may used combination of XSS attacks or weaknesses in order to target the website.

In this paper[8] Reducing attack surface corresponding to type-1 XSS attacks using the secure development life cycle principles. In a non-persistent XSS a malicious code is transferred to the application and the website owner will be unaware of the data transfer that is being happening. There will not be any encoding or validation occurs due to the presence of malicious XSS code. They have suggested input validation as their core prevention technique. The validation mechanism of a website must be string enough to verify whether the data entered by the user is fitting into the websites input criteria or not. In this way they have provided a solution. Though validation can be done we aren't sure that the user is a hacker or not. We need a firm mechanism to identify the malicious code.

The paper[9] Neutralizing SQL injection attacks using server side code modification in web applications was done. Here the input URL is taken and the parameters are extracted from it. The extracted parameters are then verified and if it contains any malicious code then the webpage rejects the URL and redirects to a warning page. As this method is only built to detect the SQL injection attacks, other type of attacks cannot be identified.

In this paper [10] A novel lightweight URL phishing detection system using SVM and similarity Index. The dataset considered here was of 2000 records where 1000 records

were datasets in order to identify the phishing websites and then tested. Volume of dataset can be increased in order to achieve a better efficiency and quick prediction rate.

In this paper[11] A second order SQL injection detection method was proposed. They have used the instruction set randomization (ISR) technique to create new SQL statements from the predefined SQL statements and detect the trusted as well as untrusted statements and hence protecting the SQL database from the attack. The randomized instruction set contains only standard set of SQL statements which might be inefficient when the attacker tries to use nested SQL queries or statements.

This paper[12] XSS detection approaches in web applications was proposed. In this paper they have proposed the prevention techniques of the following XSS attacks. They have mentioned that the input authenticity is major technique to prevent the attacks. This input data from the external user need to be verified before making it reach the server side application. Though prevention techniques were given here, there is no specific solution for the actual problem.

This paper[13] Intended approach for detection and prevention of SQL injection attacks. In this paper they have suggested a prevention mechanism where the user is suggested to use the parameterized queries in order to encapsulate the sensitive data. Though this might prevent the SQL injection attack. It is not sure that all the general users will have prior knowledge in this topic. So we need to have a solution that would be feasible for all kind of users.

CHAPTER 3

SOFTWARE REQUIREMENT ANALYSIS

3.1 SOFTWARE REQUIREMENTS

a) Python

Python is a powerful programming language and a higher – level language. Python has straightforward declarations in English that allow us to know the code without much expertise of it. With its use of a considerable whitespace, Python's structure allows readability of the code. It is also followed by indents that do not prevent the code from being analyzed or debugged. The object-oriented approach is followed by a clear overview of the logical and operational project part by the users. Projects do not have to be small.

Python Packages:

● **BeautifulSoup:** BeautifulSoup is a HTML and XML parsing Python package. It generates a parse tree for parsed pages, which is helpful in web scraping, in process of extracting HTML data. Leonard Richardson, who continues to contribute to the project and is approved by Tidelift, founded BeautifulSoup. Python 2.7 and Python 3 are both supported.
The Python library BeautifulSoup is designed for fast upturn, screen scraping projects.
It is effective with three factors:

1. To traverse, browse, and change the parse tree, Beautiful Soup includes some easy ways as well as Pythonic languages.: a document deconstructs and obtain toolkit. To type an application does not take a lot of code

2. Beautiful Soup converts incoming and outgoing documents at an instant to Unicode and UTF-8. You must not be thinking of coding except if you stipulate an encoding in the document, nor can be detected by Beautiful Soup. The actual encoding then only must be specified.

3. BeautifulSoup is built on top of popular Python parsers like lxml and html5lib, allowing you to experiment with different parsing algorithms or trade speed for versatility.

- **Pprint:** The pprint module allows arbitrarily chosen Python data structures to be 'pretty-printed' in a form that can be used as the interpreter's input. The interpretation cannot be loaded if the structures configured include objects that do not include essential Python types. This could be the case if objects like files, sockets, classes and other objects were included in the that cannot be represented as Python literals.

The formatted display holds items on a line if possible and tries to break them in multiple items if not within the width permitted. If you are to adjust the spacing constraint, build PrettyPrinter objects clearly and unambiguously

- **NumPy:** NumPy or Number Python is a library for certain array functions. It contains a n-dimensional array object useful for the whole array to apply algebra and different other mathematical formulas. The object NumPy can also be called the array NumPy. It stores information so that mathematical calculations are effectively carried out. In tabular format, NumPy Array store information, i.e., rows and columns. To use NumPy check if NumPy is or is not installed. If the NumPy package is not installed using the steps below. Go to the command prompt to setup NumPy. Enter and type the command below: "pip install numpy"

- **Pandas:** Pandas is a data manipulation and analysis programming language written in Python. It includes data structures and procedures for numerical tables and, in particular, time series manipulation. This programme is freely accessible under the three-clause BSD licence. The name derives from the term "panel data" which includes observations over multiple periods for the same people, an econometric term for data sets. The phrase "Python data analysis" itself is the reproduction of its name. Wes McKinney began building the AQR Capital pandas as a researcher between 2007 and 2010.
- **Sklearn:** Scikit-learn is a free machine learning package for Python. Python supports numerical and scientific libraries such as NumPy and SciPy, and it includes many techniques such as vector support machines, random forests, and k-neighbors. Scikit-Learn, usually known as sklearn, is Python's most popular machine learning library.

While other packages are found that work better on certain tasks, the versatility of Scikit-Learn makes it the best starting point for most ML problems. It's also an excellent beginner's library because it provides a high-level interface for a variety of activities (e.g., pre-processing data, cross-validation, etc.). This enables you to practice and comprehend the overall machine learning workflow. The project was launched as a Google Summer Code project in 2007 by David Cornopean, and has contributed numerous volunteers ever since. A team of volunteers is currently in charge of the project. Previously Scikit-learn was called Scikits.learn.

- **PyQt5:** PyQt5 is a set of Python bindings for the Qt application framework version 5 from The Qt Company. Qt is a suite of platform-independent C++ libraries and development tools for graphical user interfaces, networking, threads, regular expressions, SQL databases, SVG, OpenGL, XML, user and application settings, positioning and location services, and short-range communications (NFC and Bluetooth), web browsing, 3D animation, charts, 3D data visualization, and interacting with apps. PyQt5 implements almost 1000 of these classes as a set of Python modules. PyQt5 requires Python v3.5 or newer and runs on Windows, Linux,
  UNIX, Android, macOS, and iOS. PyQt5 also contains several utility programs

b) Interface – Spyder, Jupyter Notebook

Spyder is an IDE for integrated scientific programming in the Python Language, a cross platform open-source environment. Spyder is integrated into the scientific Python Stack with many pronounced packages, as well as NumPy, SciPy, Pandas, Ipython, Cython, and other open-source software. Spyder also includes Python Science. It is published under the MIT permit. Originally developed in 2009 by Pierre Raybaut, the group of scientific Python developers and the society have maintained and improved
Spyder constantly since 2012. Spyder's GUI is built using Qt and is designed to work with Pyqt or PySide Python bindings. QtPy a thin layer of abstraction developed by the Spyder project, which is eventually adopted by numerous other packages.

The Jupyter Notebook is a free web-based application. It is an open source in which documents containing code, text or equation can be created and shared. Jupyter is a

project that supports various programming languages by developing the Jupyter user interface.

Jupyter Notebooks is an Ipython sub-project. It is named Jupyter since it primarily supports Julia, Python, and R. Together with the kernel Ipython, Jupyter. It allows users to write their Python, R. code. There are about 100 additional kernels accessible for running the program in Jupyter.

CHAPTER 4

SOFTWARE DESIGN

Unified Modelling Language is a tool that helps a designer present his ideas about the project to his client and his developer. Modeling plays a crucial role in designing software. A poorly designed model can lead to poorly developed software. A UML system contains five different views that can be used to describe systems from various angles. Each view has a set of diagrams and components that represent the real-time objects.

A UML system contains five different views that can be used to describe systems from various angles. The real-time objects are represented by a set of diagrams and components in each view.

a. User Model View

b. Structural Model View

c. Behavioral Model View

d. Implementation Model View

4.1 USE CASE DIAGRAM

In UML, Use-case diagrams model a system's behavior and aid in capturing the system's needs. A use case diagram is a graphical representation of the interactions between a system and its actors, which are the external entities that interact with the system. It is a high-level view of the system that shows the system's functionality and the actors that interact with it.

A use case diagram consists of use cases, actors, and the relationships between them. A use case represents a specific functionality or behavior of the system, while an actor represents a user or external system that interacts with the system. The relationships between use cases and actors are represented by lines, which can be solid or dashed, depending on the type of relationship

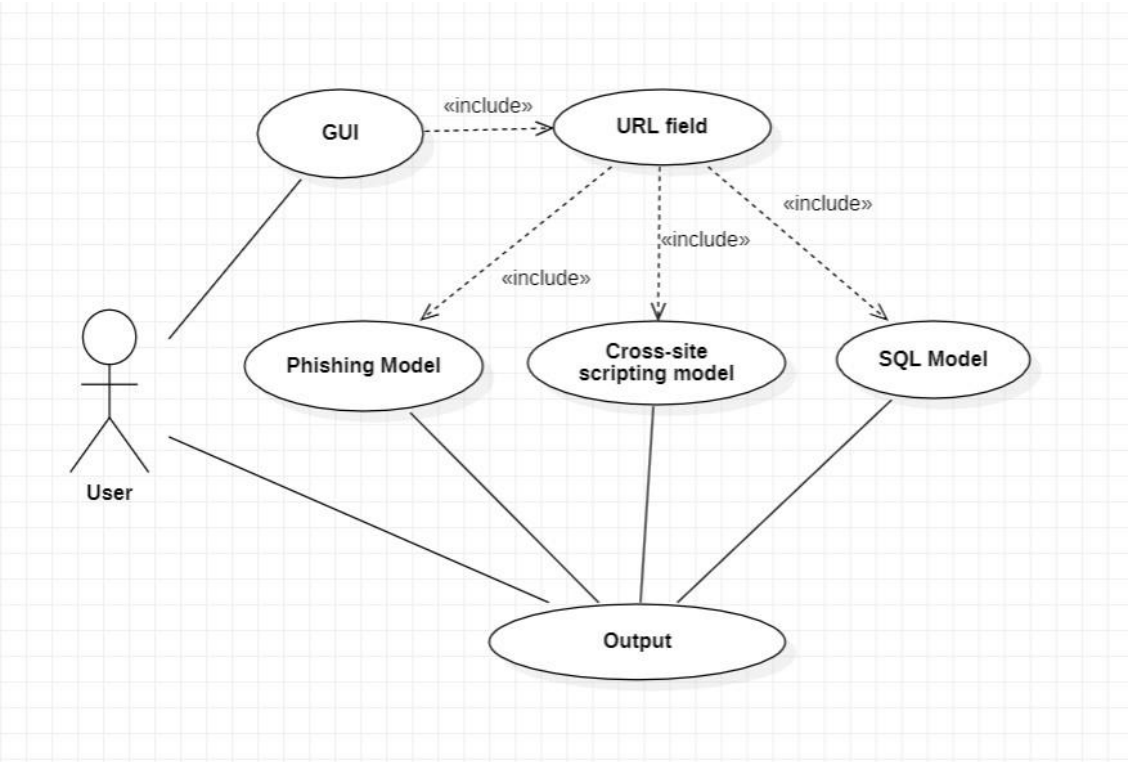

Fig 4.1: Use Case Diagram

4.2 CLASS DIAGRAM

A static view of an application is depicted in the class diagram. It depicts the several types of things found in the system and the relationships between them. A class is made up of its objects, but it can also be inherited from other classes. A class diagram is used to represent, explain, and document numerous parts of a system and to create executable software code.

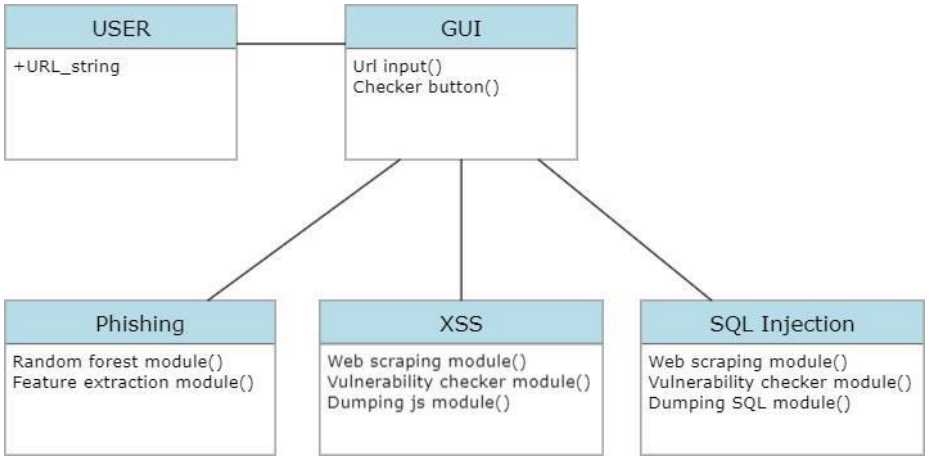

Fig 4.2: Class Diagram

4.3 SEQUENCE DIAGRAM

The Sequence Diagram depicts the interaction of several elements in an application across time. It represents the sequence of communications that objects send to one another to perform the required functionality. It consists of the lifelines which are usually parallel vertical lines. It consists of horizontal arrows which indicates the direction of the messages that are exchanged in a proper order which makes the user easy to understand. The lifeline for a given object represents a role.

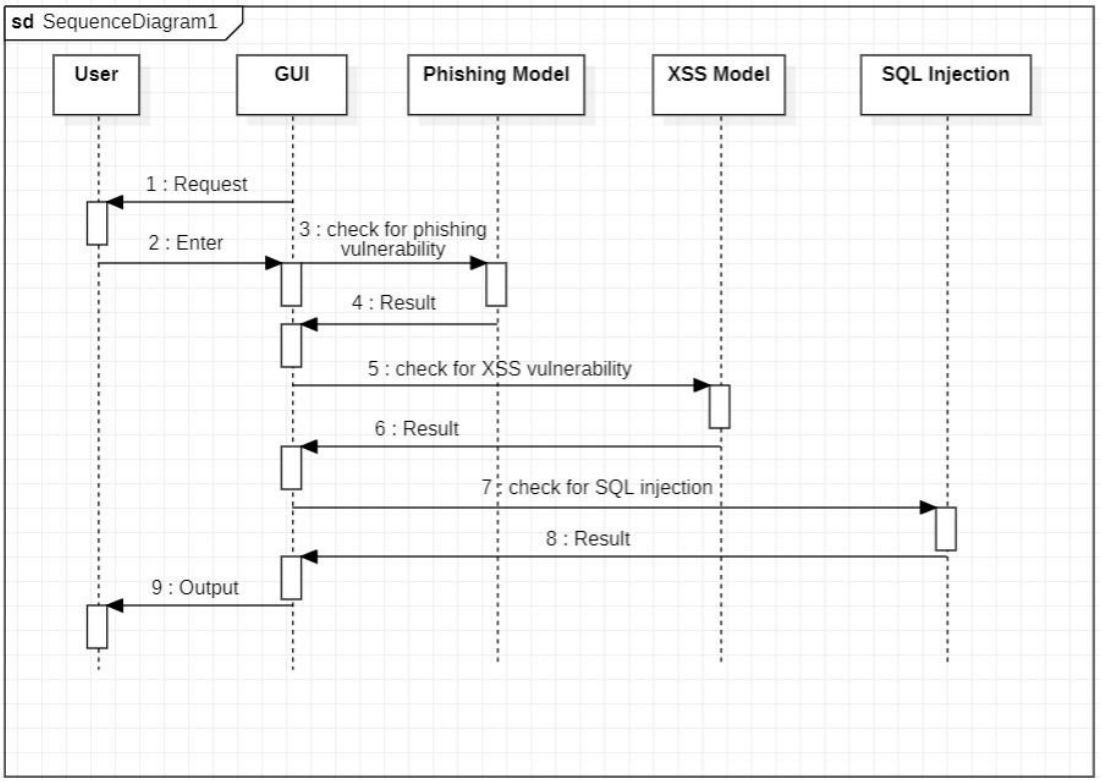

Fig 4.3: Sequence Diagram

4.4 ACTIVITY DIAGRAM

Another essential diagram in UML for describing the system's dynamic characteristics is the activity diagram. An activity diagram is a flowchart that illustrates the flow of information from one action to the next. The action can be described as a system operation.

The purpose of an activity diagram can be described as – •

Draw the activity flow of a system.

- Describe the sequence from one activity to another.
- Describe the system's parallel, branching, and concurrent flow.

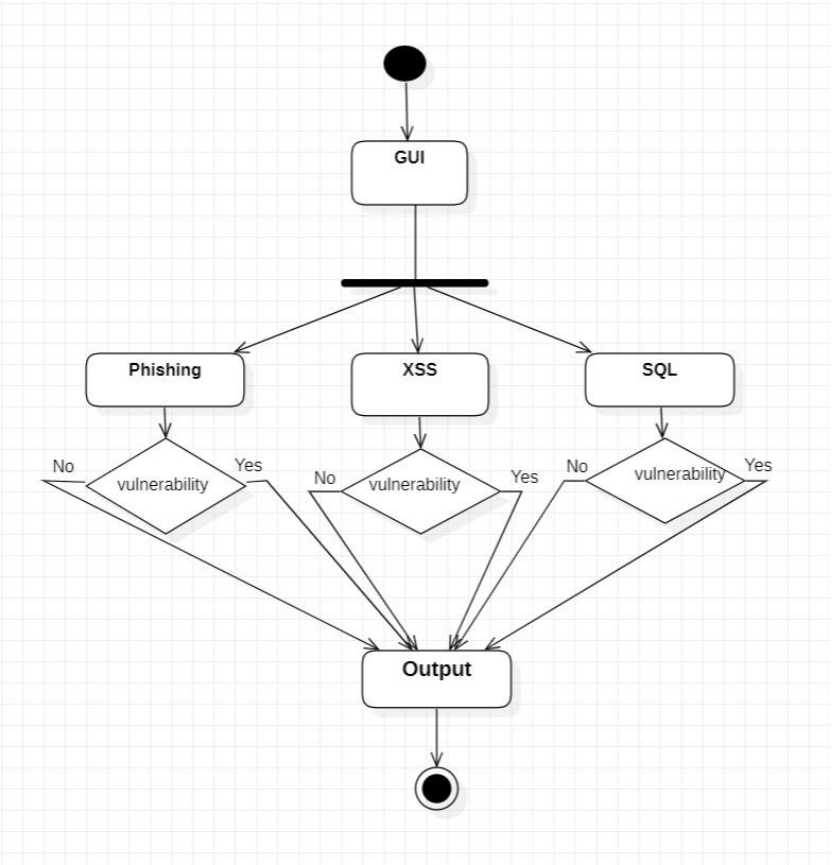

Fig 4.4: Activity Diagram

4.5 FLOW CHART

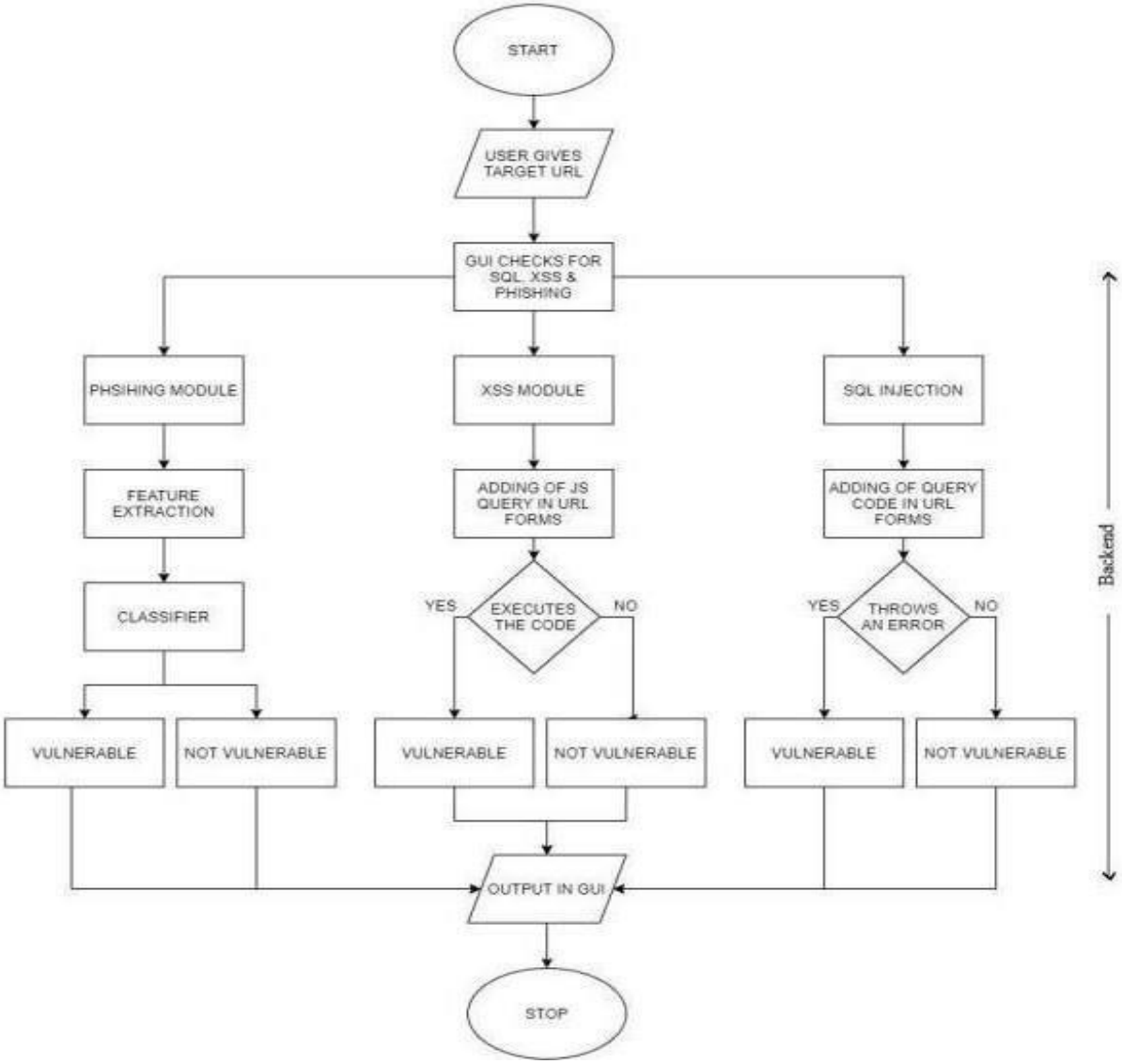

Fig 4.5: Flow diagram of the project.

CHAPTER 5

PROPOSED SYSTEM

Our project consists of three modules, for each vulnerability we have created a module. For both SQL Injection and Cross-Site Scripting detection modules, we have used the Beautiful Soup package to pull out all the existing forms that are present in the URL. In the SQL injection module, for every empty form that is present in the URL, the code incorporates a query. If the URL throws any error, it considers it as a vulnerability. Similarly, in the XSS Module instead of the query, the code incorporates client-side executable JavaScript code. We constructed a database of 2000 values for the phishing module and divided it into two portions (training and testing). We have chosen certain feature extractions which help us in increasing the accuracy of detection. For each feature, we have created a new column in the existing dataset and these features act as decision trees in building the random forest model. As we know that random forest algorithm takes all these decision trees (extraction features) and merges them to get more accurate and stable results. We used Jupyter Notebook, Matplotlib, and NumPy packages for building the random forest model and incorporated this trained model in the GUI which is created in Jupyter Notebook. The user just needs to run the GUI module and should have to provide the URL as an input and in the backend, all these modules run and provide the user with an output in the GUI.

In practical, four modules are involved in this project:

1. Detection of SQL Injection (Server side)
2. Detection of XXS vulnerability (Client side)
3. Detection of Phishing vulnerability (Social engineering)
4. Building a Graphical Interface (GUI).

5.1 Detection of SQL Injection (Server side)

When a malicious user input is accepted by an application, it is subsequently used as part of a SQL statement to query a backend database. To modify the query structure, an attacker can inject SQL control characters and command keywords (e.g., single quote ('), double quote ("), equal (=), comment (- -), etc.). When these control characters are combined with typical SQL commands (e.g., SELECT, FROM, DELETE, etc.) data components from a backend database server can be accessed or retrieved. A successful attack is carried out by an attacker inserting malicious code into the program using a SQL statement. An attacker can modify or access a backend database when malicious SQL commands are executed against it.

We have installed packages for extracting URL forms before starting this module. The packages BeautifulSoup, URL, Request and pprint have been used. We need to remove forms and text boxes from the web application for SQL Injection detection. So, we have written a function that automatically receives an input and gives extracted forms as output. We used the BeautifulSoup library to extract all form tags from HTML. This extraction takes place in the function get allforms(). A single form tag object argument is added in the other function getting formdetails() and it only analyses applicable information. They target URL, GET, POST, and input field attributes.

If a website has any SQL injection vulnerability it considers the user given SQL statement as executable input. Whenever a user gives a statement (like or select statements). The website throws an error based on the type of database version they use. So, in simple words if the website throws this kind of error when user gives an SQL statement then that website is vulnerable to SQL injection.

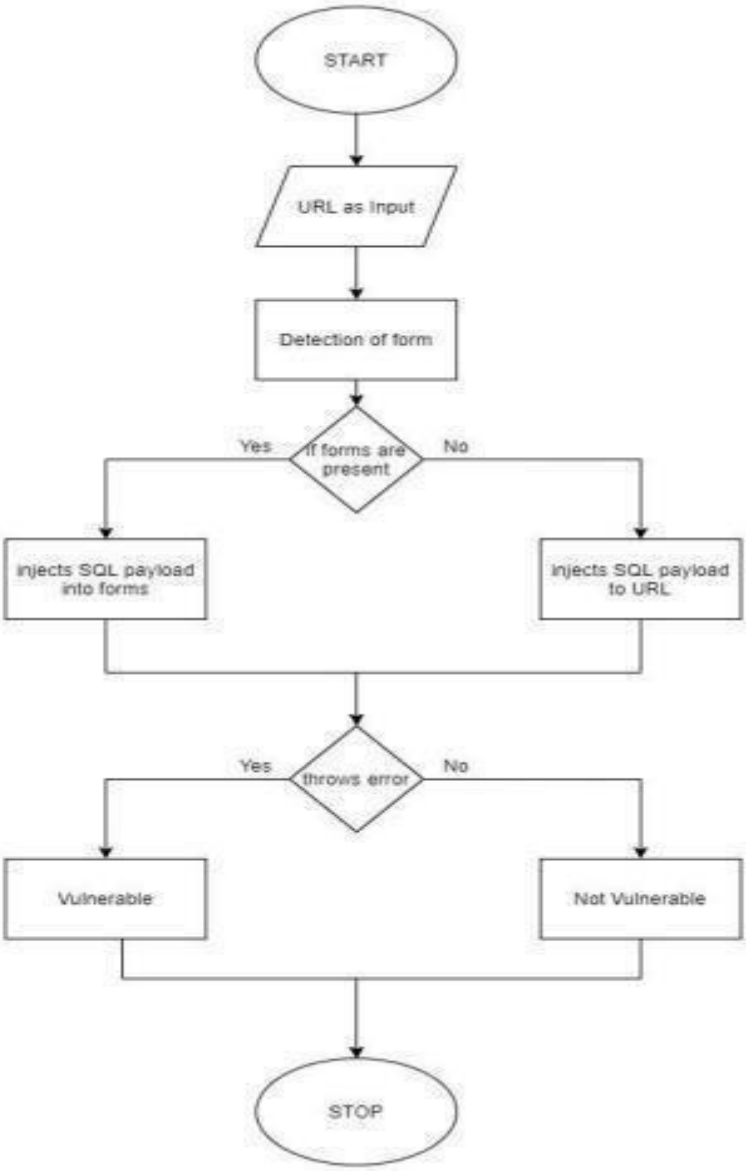

Fig 5.1: Flow Diagram of SQL Injection.

```
errors = {
    # MySQL
    "you have an error in your sql syntax;",
    "warning: mysql",
    # SQL Server
    "unclosed quotation mark after the character string",
    # Oracle
    "quoted string not properly terminated",
}
```

Fig 5.2: List of errors that are thrown by SQL vulnerable websites

We explicitly added all the errors that occur in SQL vulnerable websites. Now a new function is vulnerable (URL) added which takes URL as an argument and scrapes all the forms from the website and stores them in a temporary variable. It pushes the SQL payloads into all the forms using formdetails() function and reads the error thrown by the website. Then it compares the error with the explicitly stored errors. If both error statements match, then it is considered as SQL Injection vulnerable. But few websites do not have any forms but still, they are vulnerable to SQL Injection. For such kinds of URLs, this same function adds payload "single quote (')" at the end of the URL. Then they may throw the same problems, and it discovers the SQL Injection vulnerability by comparing the errors.

5.2 Detection of XSS vulnerability (Client side)

An XSS vulnerability occurs if web applications take user data and dynamically add this data to websites without evaluating the information properly before. An attacker may execute random commands and random contents in the browser of the victim by using XSS vulnerabilities. A successful XSS attack results in an attacker modifying the victim's browser or online application. While XSS is being enabled through vulnerable web-based pages, users are the victims of an XSS attack, not the application itself. The potential for an XSS vulnerability is because in the context of the victim session, the malicious code executes allowing the attacker to bypass normal security limitations.

We have installed packages for extracting URL forms before starting this module. The packages BeautifulSoup, URL, Request and pprint have been used. For an XSS vulnerability, we need to remove forms and text boxes from the website. So, we have written a function that automatically receives an input and gives extracted forms as output. We used the BeautifulSoup library to extract all form tags from HTML. This extraction takes place in the function get allforms(). A single form tag object argument is added in the other function getting formdetails() and only analyses applicable information. Those are the target URL, GET, POST, and input field attributes.

The function get_formdetails() collects all the details of forms (text, search, etc.) from the website. After gathering all the information based on the website structure the submission takes place either in the get or post method. The main function collects all HTML forms and then prints the detected range of forms. It then evaluates all the forms and presents the forms with the JavaScript code for the value of all text and search fields. The injection

of JavaScript code is done by using the above get_formdetails() function and submits the JavaScript code. It is considered vulnerable if the inserted JavaScript code is successfully executed.

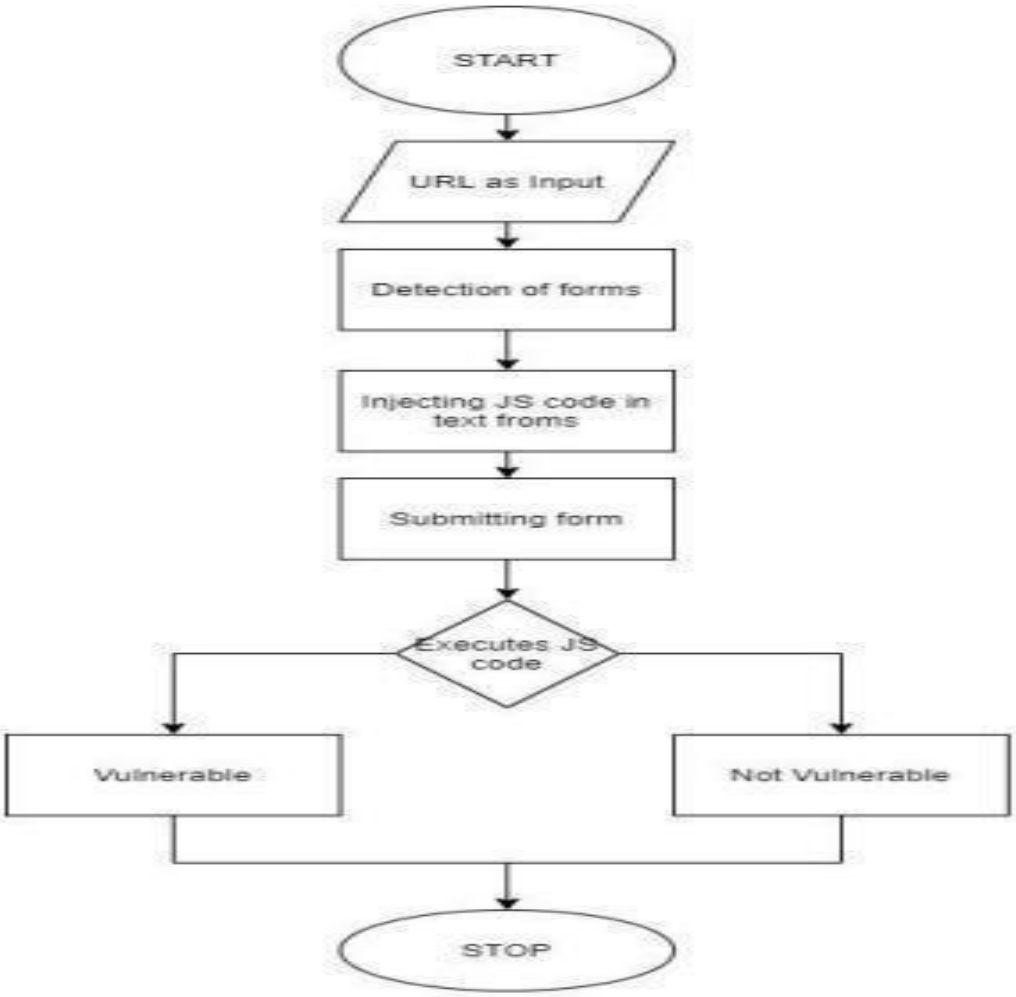

Fig 5.3: Flow chart of XSS vulnerability

```
js_script = "<Script>alert('hi')</script>"
# returning value
is_vulnerable = False
# iterate over all forms
for form in forms:
    form_details = get_form_details(form)
    content = submit_form(form_details, url, js_script).content.decode()
    if js_script in content:
        print(f"[+] XSS Detected on {url}")
        #print(f"[*] Form details:")
        pprint(form_details)
        is_vulnerable = True
        return("XSS: Vulnerability detected")
```

Fig 5.4: Code of injecting JavaScript code into forms.

We can say that if a website executes the JavaScript snippet in the form and gives output to the user like “alert ()” statements. Then the website has XSS vulnerability.

5.3 Detection of Phishing vulnerability (Social Engineering)

An attacker can send thousands of fraudulent messages, even if only a small proportion of the recipients fall for fraud, to steal significant information and money. Hackers design phishing pages to imitate a spoofed organization’s actual websites. The websites appear legitimate using the same phrases, types, logos, and signatures. Furthermore, attackers often use urgency to persuade users to act. By providing this pressure, the user becomes less diligent and more prone to errors. Links within messages look like their legal colleagues, but typically have an incorrect domain name or additional subdomains. My.edu/renewal URLs, for example, have been changed to my.edu/renewal.com university.

Similarity between these two addresses leave the illusion that there is a safe link, which lessens the awareness of the attack.

In this module, we used a dataset of approximately 2000 web URLs. We split the URL in half for training and testing, with 80 percent going to training and 20 percent going to testing. The training dataset accounts for 80% of the total, whereas the testing dataset accounts for 20%. A few features must be considered while detecting a phishing website. The features are

1. Having a long URL that hides the suspicious part of the URL
2. URL having “@” symbol
3. Having a “//” symbol in the URL, which redirects the page to a malicious one.
4. Adding Prefix or Suffix Separated by “-’’ to the Domain
5. Sub-Domain and Multi Sub-Domains

These are the features that can indicate whether a website is malicious. Going through a detailed explanation of the features.

1. If the URL is bigger than or equal to 54 characters, it is considered phishing. 0 --- indicates legitimate

1 --- indicates Phishing

0 --- indicates Suspicious

2. Using the “@” symbol in the URL, the browser overlooks all the above “@” symbol, and the address usually follows the “@” symbol.

IF {URL Having @ Symbol→ Phishing Otherwise→ Legitimate}

0 --- indicates legitimate

1 --- indicates Phishing

3.The existence of “//” within the URL path means that the user will be redirected to another website. An example of such URLs is: “http://www.legitimate.com//http://www.phishing.com”. We examine the location where the “//” appears. We discovered that if the URL begins with "HTT," the "//" should appear in the sixth position. If the URL contains "HTTPS," the "//" should appear in the seventh position.

4.IF {The Position of the Last Occurrence of “//” in the URL > 7→ Phishing Otherwise→ Legitimate

0 --- indicates legitimate

0 --- indicates Phishing

5. In genuine URLs the hyphen (-) symbol was hardly used. Phishers are prone to add prefixes or suffixes split by to make users believe they are dealing with a genuine webpage (-). For example, http://www.confirmpay-paypalmoney.com/.

IF {Domain Name Part Includes (−) Symbol → Phishing Otherwise → Legitimate

0 □ indicates phishing 0 □ indicates legitimate

6. There are two dots in the URL for a legitimate URL link, because “www.” Can be neglected. The URL is categorized as “suspicious” if the number of points equaled three since it is divided by one subdomain. But if the points exceed three, they are categorized as “Phishy” because they will be subdomains.

0 --- indicates legitimate 1 --- indicates Phishing 2 --- indicates Suspicious

As we know, Random forests combine all the decision trees to produce a stable and precise result. Here these five features act as individual decision-making trees. So, in the random forest process, we employ these five features to achieve the result. The findings show if the URL is phishing.

5.4 Building Graphical User Interface

GUI was developed using python and PyQt5 package. We title on the top using qt widgets module and we also incorporated Qpixmap. For user input, we made an input box in the center using Qlabel. A button is made to toggle the running of all the functions. Finally, an output box is created on the bottom using Qlabel and for cursor elements we used Qcursor.

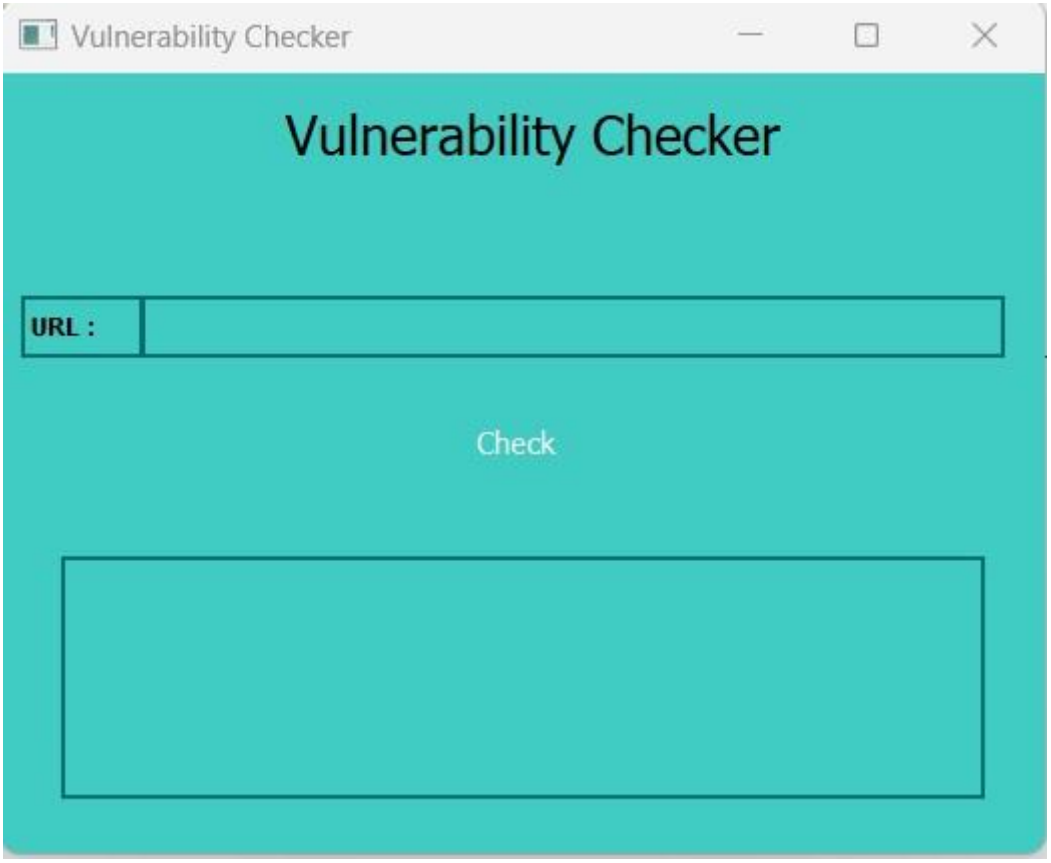

Fig 5.5: Structure of Graphical User Interface

The main reason to create this GUI is to link all the three vulnerabilities function and get the output in a single place to improve the user experience. When a user enters a URL as an input and clicks the check button in the backend, all three vulnerability detection modules are enabled with this URL as an argument. Each module's output is saved in temporary variables. In the end, after we receive all the output from these modules, we print them in the output window of the GUI. Whenever the user gives an incorrect URL, a message is prompted saying please enter the correct URL. URLs without “HTTP” or which are not working are considered incorrect URLs.

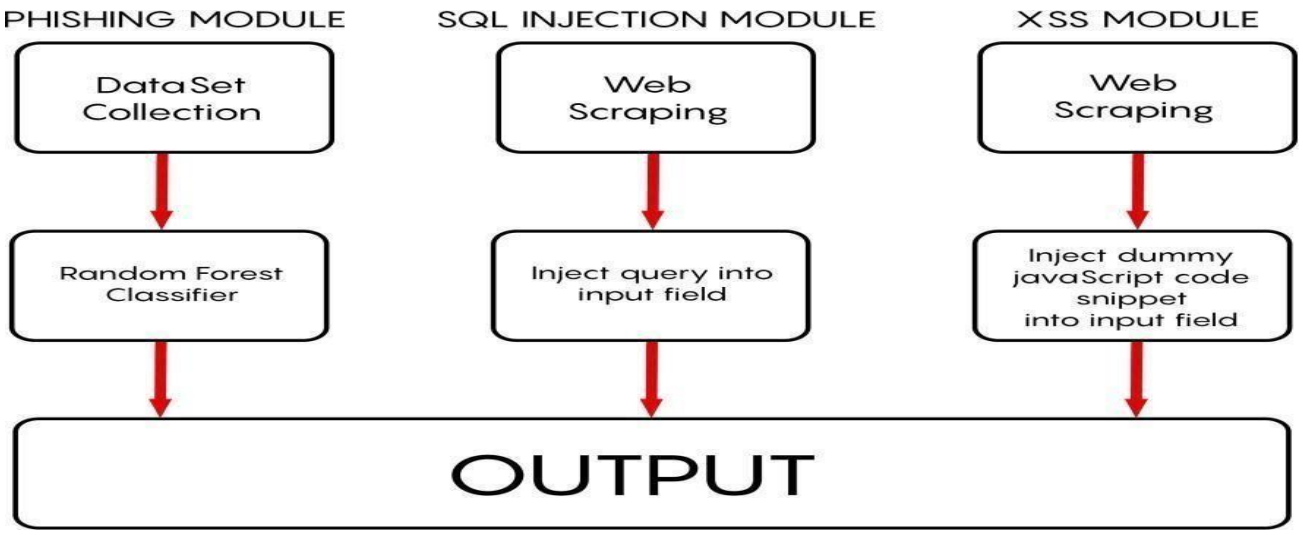

Fig 5.6: Tier Architecture

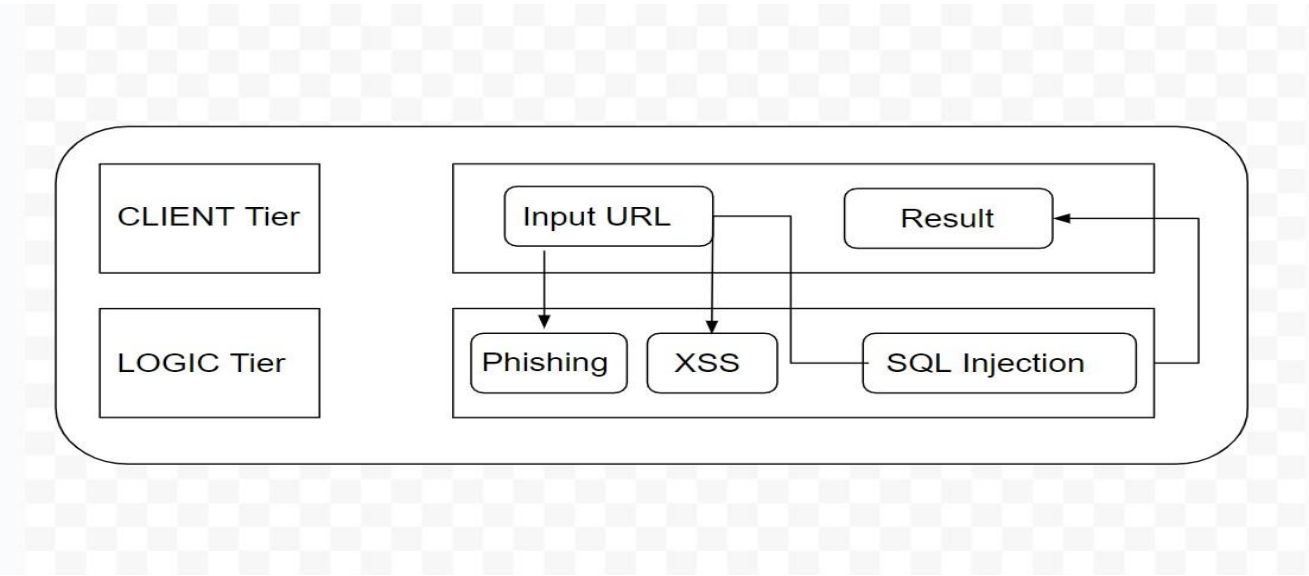

Fig 5.7: Workflow

CHAPTER 6

IMPLEMENTATION

Phishing module :

```
import pandas as pd
#import numpy as np

import rf_model


class feature_extractor:
        def __init__(self,url:str):

            self.input_url = url

        def long_url(self,l):
            """This function is defined in order to differntiate website based on the length of the URL"""
            l= str(l)
            if len(l) < 54:
                return 0
            elif len(l) >= 54 and len(l) <= 75:
                return 2
            return 1

        def have_at_symbol(self,l):
            """This function is used to check whether the URL contains @ symbol or not"""
            if "@" in str(l):
                return 1
            return 0

        def redirection(self,l):
            """If the url has symbol(//) after protocol then such URL is to be classified as phishing """
            if "//" in str(l):
                return 1
            return 0

        def prefix_suffix_seperation(self,l):
            """seprate prefix and suffix"""
            if '-' in str(l):
                return 1
            return 0

        def sub_domains(self,l):
            """check the subdomains"""
            l= str(l)
            if l.count('.') < 3:
                return 0
            elif l.count('.') == 3:
                return 2
            return 1


        def extract(self):
            try:

                print("in script 2")
                input_data = [{"URL":self.input_url}]
                print('input taken')
```

Fig 6.1: Phishing module- 1

```
            return 0
        elif l.count('.') == 3:
            return 2
        return 1


    def extract(self):
        try:

            print("in script 2")
            input_data = [{"URL":self.input_url}]
            print('input taken')
            temp_df = pd.DataFrame(input_data)
            print("dataframe created")
            #expand argument in the split method will give you a new column
            seperation_of_protocol = temp_df['URL'].str.split("://",expand = True)
            print("step 1 done")
            #split(seperator,no of splits according to seperator(delimiter),expand)
            seperation_domain_name = seperation_of_protocol[1].str.split("/",1,expand = True)
            print("step 2 done")
            #renaming columns of data frame
            seperation_domain_name.columns=["domain_name","address"]
            print("step 3 done")
            #Concatenation of data frames
            splitted_data = pd.concat([seperation_of_protocol[0],seperation_domain_name],axis=1)
            print("step 4 done")

            splitted_data.columns = ['protocol','domain_name','address']
            print("step 5 done")

            #splitted_data['is_phished'] = pd.Series(temp_df['Target'], index=splitted_data.index)
            #print("step 6 done")

            """feature extraction starts here"""
            #Applying the above defined function in order to divide the websites into 3 categories
            splitted_data['long_url'] = temp_df['URL'].apply(self.long_url)
            print("feature extra 1")
            splitted_data['having_@_symbol'] = temp_df['URL'].apply(self.have_at_symbol)
            print("feature extra 2")
            splitted_data['redirection_//_symbol'] = seperation_of_protocol[1].apply(self.redirection)
            print("feature extra 3")
            splitted_data['prefix_suffix_seperation'] = seperation_domain_name['domain_name'].apply(self.prefix_suffix_seper
            print("feature extra 4")
            splitted_data['sub_domains'] = splitted_data['domain_name'].apply(self.sub_domains)
            print("feature extra 5")
            #splitted_data.to_csv(r'dataset3.csv',header= True)



            return rf_model.predictor(splitted_data)
        except:
            return ("Please enter Correct URL in the format (http:// or https://)")
```

Fig 6.2: Phishing module- 2

Graphical output module :

```
import sys
from PyQt5.QtWidgets import QLabel
from PyQt5.QtGui import QPixmap
from PyQt5 import QtGui, QtCore,QtWidgets
from PyQt5.QtGui import QCursor
import sql_injection_scanner
import xss_scanner
import phish_feature_extractor
import rf_model


class Ui_Sql_detector(object):
        def setupUi(self,window):
            window.setObjectName("window")
            window.resize(521, 389)
            icon = QtGui.QIcon("bug.jpg")
            window.setWindowIcon(icon)
            #window.setWindowIcon(QIcon('bug.png'))

            self.centralwidget = QtWidgets.QWidget(window)
            self.centralwidget.setObjectName("centralwidget")

            self.icon = QLabel()

            # loading image
            self.pixmap = QPixmap('bug.png')

            # adding image to label
            self.icon.setPixmap(self.pixmap)

            # Optional, resize label to image size
            self.icon.resize(self.pixmap.width(),
                             self.pixmap.height())
            self.icon.setObjectName("icon")
            # show all the widgets
            #self.label.show()



            """check button code and its connectivity to button_click function"""
            self.check_button = QtWidgets.QPushButton(self.centralwidget)
```

Fig 6.3: GUI module- 1

```
        """check button code and its connectivity to button_click function"""
        self.check_button = QtWidgets.QPushButton(self.centralwidget)
        self.check_button.setGeometry(QtCore.QRect(210, 170, 93, 28))
        self.check_button.setCursor(QCursor(QtCore.Qt.PointingHandCursor))
        self.check_button.setStyleSheet(
        "*{border: 2px solid '#16121';" +
        "border-radius: 75px;" +
        "font-size: 15px;" +
        "color: 'white';" +
        "padding: 15px 0;" +
        "margin: 100px 200px;}" +
        "*:hover{background: '#16121';}")
        self.check_button.setObjectName("check_button")
        self.check_button.clicked.connect(self.button_click)

        """url input section"""
        self.url_input = QtWidgets.QLineEdit(self.centralwidget)
        self.url_input.setGeometry(QtCore.QRect(70, 111, 431, 31))
        self.url_input.setStyleSheet(
        "border: 2px solid '#046e6e';" +
        "color: 'black';" + "font-size: 12px;" )
        self.url_input.setObjectName("url_input")

        self.label = QtWidgets.QLabel(self.centralwidget)
        self.label.setGeometry(QtCore.QRect(10, 111, 61, 31))
        self.label.setStyleSheet(
        "border: 2px solid '#046e6e';" +
        "color: 'black';" + "font-size: 12px;")
        self.label.setObjectName("label")

        """output message"""
        self.output_text = QtWidgets.QTextEdit(self.centralwidget)
        self.output_text.setGeometry(QtCore.QRect(30, 241, 461, 121))
        self.output_text.setStyleSheet(
        "border: 2px solid '#046e6e';" +
        "color: 'black';" + "font-size: 15px;")
        self.output_text.setObjectName("output_text")

        self.label_2 = QtWidgets.QLabel(self.centralwidget)
        self.label_2.setGeometry(QtCore.QRect(110, 10, 311, 41))
```

Fig 6.4: GUI module- 2

```
        QtCore.QMetaObject.connectSlotsByName(window)

    def retranslateUi(self, window):
        _translate = QtCore.QCoreApplication.translate
        window.setWindowTitle(_translate("window", "Vulnerability Checker"))

        self.check_button.setText(_translate("window", "Check"))
        self.label.setText(_translate("window", "<html><head/><body><p><span style=\" font-si
        self.label_2.setText(_translate("window", "<html><head/><body><p align=\"center\"><sp


    def button_click(self):

        #passing URL as input
        url = self.url_input.text()

        #calling XSS scanner function
        xss_str= xss_scanner.scan_xss(url)

        #calling SQL Injection scanner functions
        sql_str = sql_injection_scanner.scan_sql_injection(url)

        #calling Phishing functions
        phish_obj = phish_feature_extractor.feature_extractor(url)
        phish_str = phish_obj.extract()

        #to view the output
        self.output_text.append("Testing url : {}\n{}\n{}\n{}\n\n".format(url,phish_str,xss_s

if __name__ == "__main__":

    app = QtWidgets.QApplication(sys.argv)
    window = QtWidgets.QMainWindow()
    window.setStyleSheet("background: #41ccc3; ")

    ui = Ui_Sql_detector()
    ui.setupUi(window)
    window.show()
    sys.exit(app.exec())
```

Fig 6.5: GUI module- 3

SQL Injection module :

```python
import requests
from bs4 import BeautifulSoup as bs
from urllib.parse import urljoin
from pprint import pprint

# initialize an HTTP session & set the browser
s = requests.Session()
s.headers["User-Agent"] = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, l



def get_all_forms(url):
    """Given a `url`, it returns all forms from the HTML content"""
    soup = bs(s.get(url).content, "html.parser")
    return soup.find_all("form")


def get_form_details(form):
    """
    This function extracts all possible useful information about an HTML `form`
    """
    details = {}
    # get the form action (target url)
    try:
        action = form.attrs.get("action").lower()
    except:
        action = None
    # get the form method (POST, GET, etc.)
    method = form.attrs.get("method", "get").lower()
    # get all the input details such as type and name
    inputs = []
    for input_tag in form.find_all("input"):
        input_type = input_tag.attrs.get("type", "text")
        input_name = input_tag.attrs.get("name")
        input_value = input_tag.attrs.get("value", "")
        inputs.append({"type": input_type, "name": input_name, "value": input_value})
    # put everything to the resulting dictionary
    details["action"] = action
    details["method"] = method
    details["inputs"] = inputs
```

Fig 6.6: SQL Injection module - 1

```python
    return details



def is_vulnerable(response):
    """A simple boolean function that determines whether a page
    is SQL Injection vulnerable from its `response`"""
    errors = {
        # MySQL
        "you have an error in your sql syntax;",
        "warning: mysql",
        # SQL Server
        "unclosed quotation mark after the character string",
        # Oracle
        "quoted string not properly terminated",
    }
    for error in errors:
        # if you find one of these errors, return True
        if error in response.content.decode().lower():
            return True
    # no error detected
    return False

def scan_sql_injection(url):
    try:
        # test on URL
        for c in "\"'":
            # add quote/double quote character to the URL
            new_url = f"{url}{c}"
            print("[!] Trying", new_url)
            # make the HTTP request
            res = s.get(new_url)
            if is_vulnerable(res):
                # SQL Injection detected on the URL itself,
                # no need to preceed for extracting forms and submitting them
                print("vuln deteected")
                return("SQL Injection: Vulnerability detected")
            elif is_vulnerable(res):
                print("vuln not det")
                return("SQL Injection: Vulnerability NOT detected")
```

Fig 6.7: SQL Injection Module - 2

```python
        # test on HTML forms
        forms = get_all_forms(url)
        print(f"[+] Detected {len(forms)} forms on {url}.")
        for form in forms:
            form_details = get_form_details(form)
            for c in "\"'":
                # the data body we want to submit
                data = {}
                for input_tag in form_details["inputs"]:
                    if input_tag["type"] == "hidden" or input_tag["value"]:
                        # any input form that is hidden or has some value,
                        # just use it in the form body
                        try:
                            data[input_tag["name"]] = input_tag["value"] + c
                        except:
                            pass
                    elif input_tag["type"] != "submit":
                        # all others except submit, use some junk data with special character
                        data[input_tag["name"]] = f"test{c}"
                # join the url with the action (form request URL)
                url = urljoin(url, form_details["action"])
                if form_details["method"] == "post":
                    res = s.post(url, data=data)
                elif form_details["method"] == "get":
                    res = s.get(url, params=data)
                # test whether the resulting page is vulnerable
                if is_vulnerable(res):
                    print("[+] SQL Injection vulnerability detected, link:", url)
                    print("[+] Form:")
                    pprint(form_details)
                    return ("SQL Injection: Vulnerability detected")

        print("not vulnerable")
        return ("SQL Injection: Vulnerability Not detected")
    except:
        return ("")
```

Fig 6.8: SQL Injection Module - 3

XSS Module :

```
import requests
from pprint import pprint
from bs4 import BeautifulSoup as bs
from urllib.parse import urljoin


def get_all_forms(url):
    """Given a `url`, it returns all forms from the HTML content"""
    soup = bs(requests.get(url).content, "html.parser")
    return soup.find_all("form")


def get_form_details(form):
    """
    This function extracts all possible useful information about an HTML `form
    """
    details = {}
    # get the form action (target url)
    action = form.attrs.get("action").lower()
    # get the form method (POST, GET, etc.)
    method = form.attrs.get("method", "get").lower()
    # get all the input details such as type and name
    inputs = []
    for input_tag in form.find_all("input"):
        input_type = input_tag.attrs.get("type", "text")
        input_name = input_tag.attrs.get("name")
        inputs.append({"type": input_type, "name": input_name})
    # put everything to the resulting dictionary
    details["action"] = action
    details["method"] = method
    details["inputs"] = inputs
    return details


def submit_form(form_details, url, value):
    """
    Submits a form given in `form_details`
    Params:
        form_details (list): a dictionary that contain form information
        url (str): the original URL that contain that form
        value (str): this will be replaced to all text and search inputs
    Returns the HTTP Response after form submission
```

Fig 6.9: XSS Module - 1

```python
    # construct the full URL (if the url provided in action is relative)
    target_url = urljoin(url, form_details["action"])
    # get the inputs
    inputs = form_details["inputs"]
    data = {}
    for input in inputs:
        # replace all text and search values with `value`
        if input["type"] == "text" or input["type"] == "search":
            input["value"] = value
        input_name = input.get("name")
        input_value = input.get("value")
        if input_name and input_value:
            # if input name and value are not None,
            # then add them to the data of form submission
            data[input_name] = input_value

    if form_details["method"] == "post":
        return requests.post(target_url, data=data)
    else:
        # GET request
        return requests.get(target_url, params=data)


def scan_xss(url):
    try:
        """
        Given a `url`, it prints all XSS vulnerable forms and
        returns True if any is vulnerable, False otherwise
        """
        # get all the forms from the URL
        forms = get_all_forms(url)
        print(f"[+] Detected {len(forms)} forms on {url}.")
        js_script = "<Script>alert('hi')</script>"
        # returning value
        is_vulnerable = False
        # iterate over all forms
        for form in forms:
            form_details = get_form_details(form)
            content = submit_form(form_details, url, js_script).content.decode()
            if js_script in content:
                print(f"[+] XSS Detected on {url}")
                #print(f"[*] Form details:")
```

Fig 6.10: XSS Module - 2

```python
def scan_xss(url):
    try:
        """
        Given a `url`, it prints all XSS vulnerable forms and
        returns True if any is vulnerable, False otherwise
        """
        # get all the forms from the URL
        forms = get_all_forms(url)
        print(f"[+] Detected {len(forms)} forms on {url}.")
        js_script = "<Script>alert('hi')</script>"
        # returning value
        is_vulnerable = False
        # iterate over all forms
        for form in forms:
            form_details = get_form_details(form)
            content = submit_form(form_details, url, js_script).content.decode()
            if js_script in content:
                print(f"[+] XSS Detected on {url}")
                #print(f"[*] Form details:")
                pprint(form_details)
                is_vulnerable = True
                return("XSS: Vulnerability detected")
                # won't break because we want to print available vulnerable forms
            else:
                print("no")
                #is_vulnerable= False
                return("XSS: Vulnerability not detected")
                break
        return ("XSS: Vulnerability not detected")
    except:
        return ("")
```

Fig 6.11: XSS Module - 3

CHAPTER 7

TESTING

The application was tested for the correctness against certain legal websites such as Yahoo and Twitter. The testing as done with the URL

Testing Yahoo website

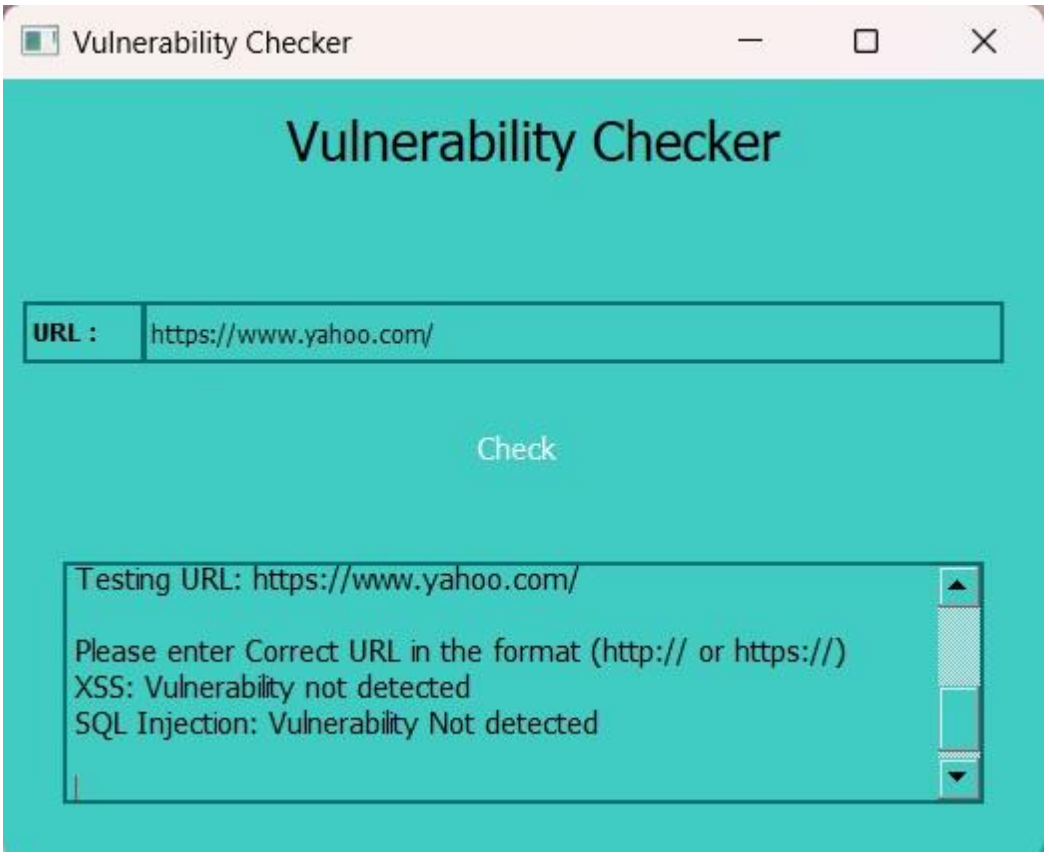

Fig 7.1: Testcase - 1

Testing Twitter website

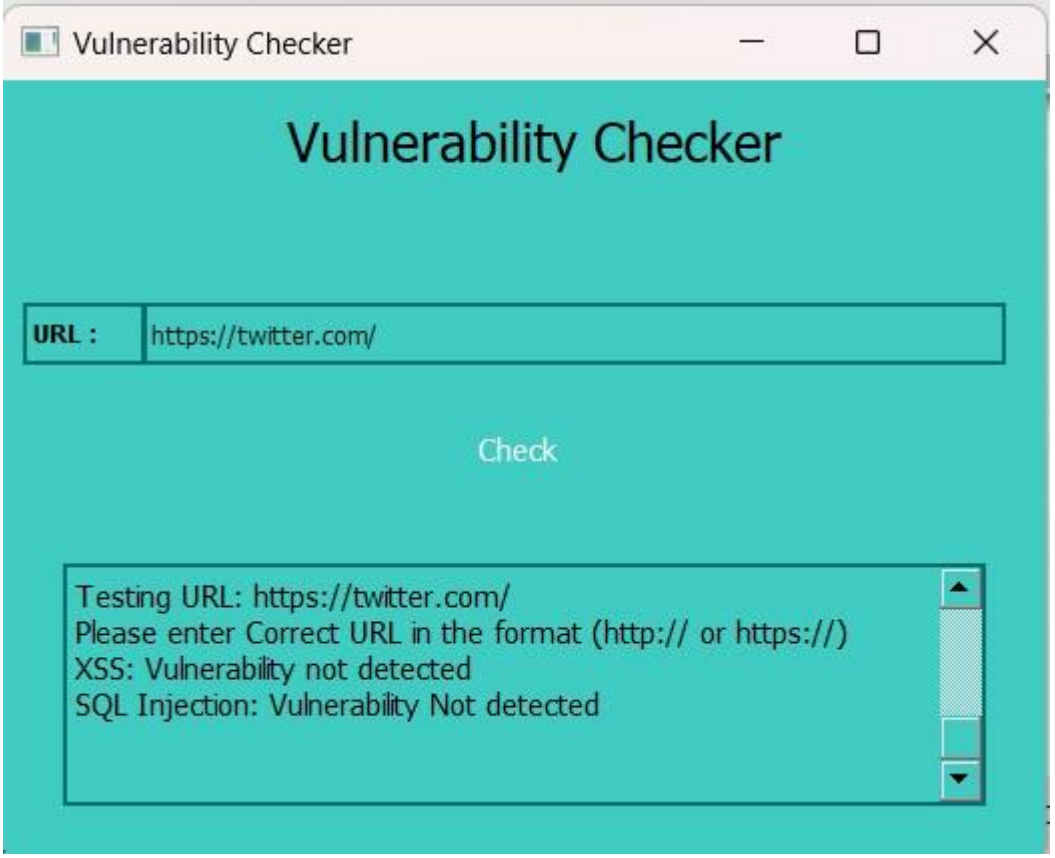

Fig 7.2: Testcase - 2

The application as tested for the correctness against certain illegal websites. The testing was done with the URL

Testing http://testphp.vulneb.com/artists.php?artist=2

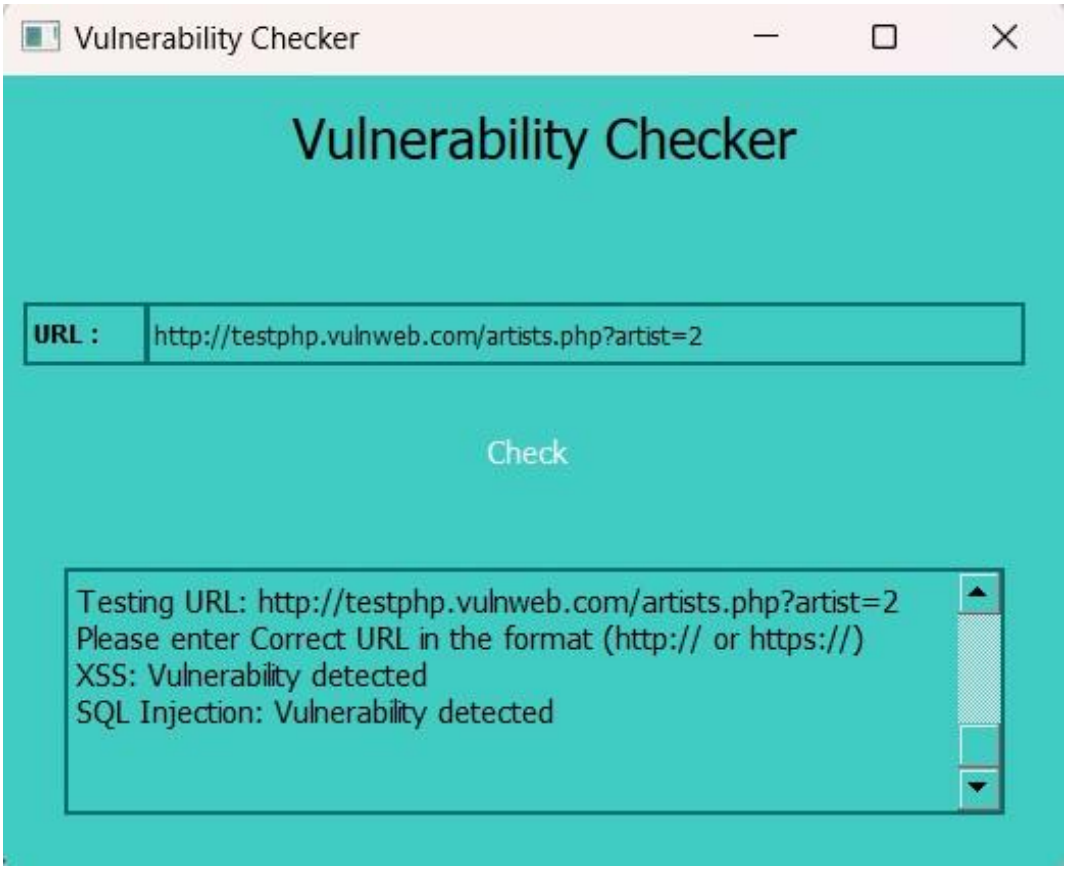

Fig 7.3: Testcase - 3

CHAPTER 8

RESULTS

In this part, the results of implementing the proposed system are presented.

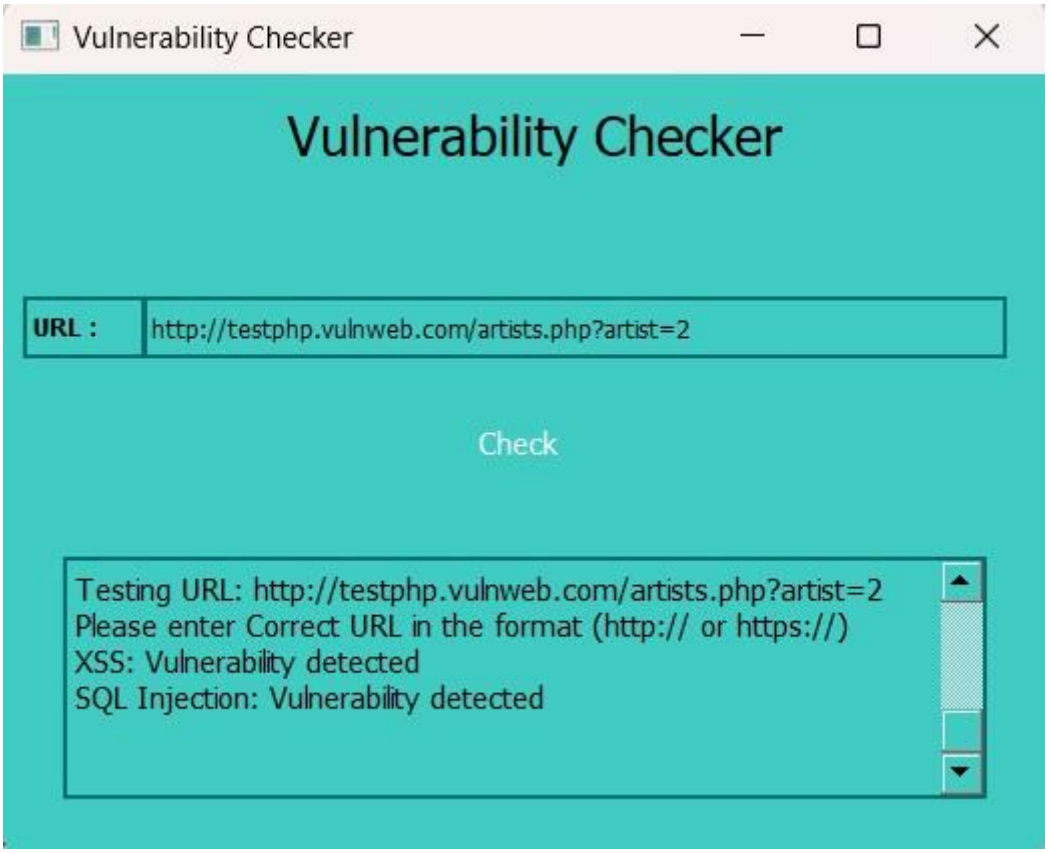

Fig 8.1: Output - 1

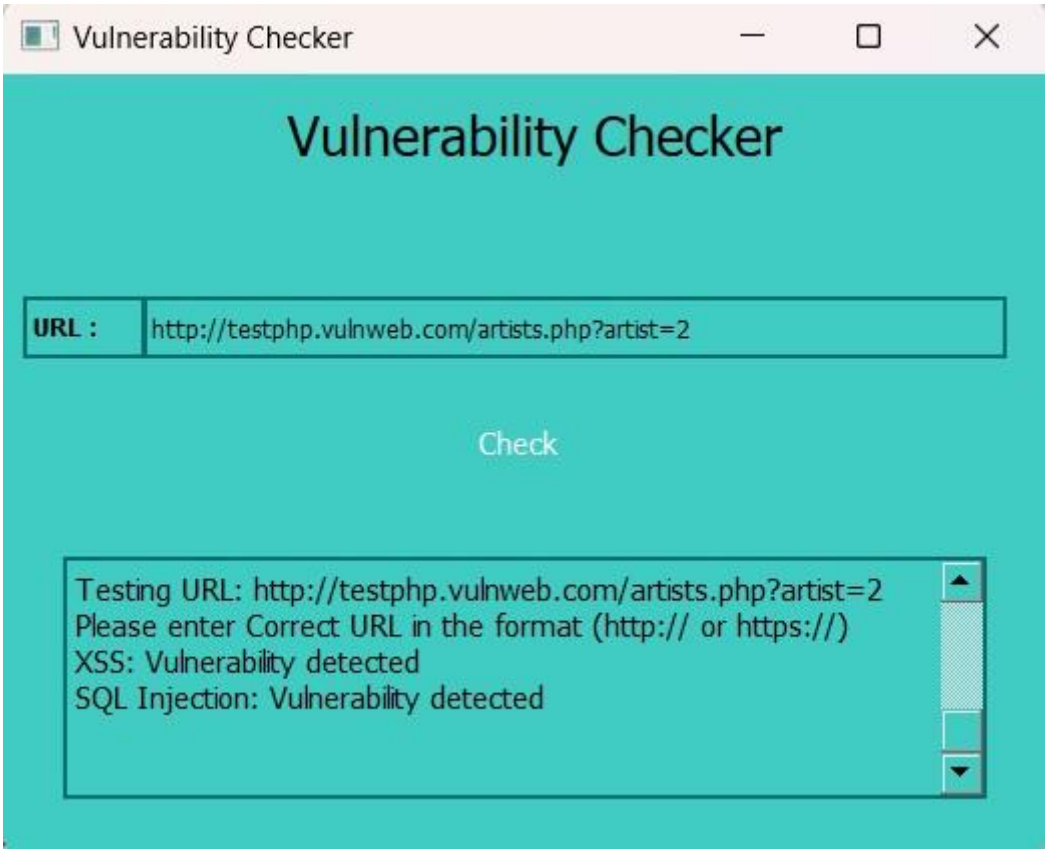

Fig 8.2: Output - 2

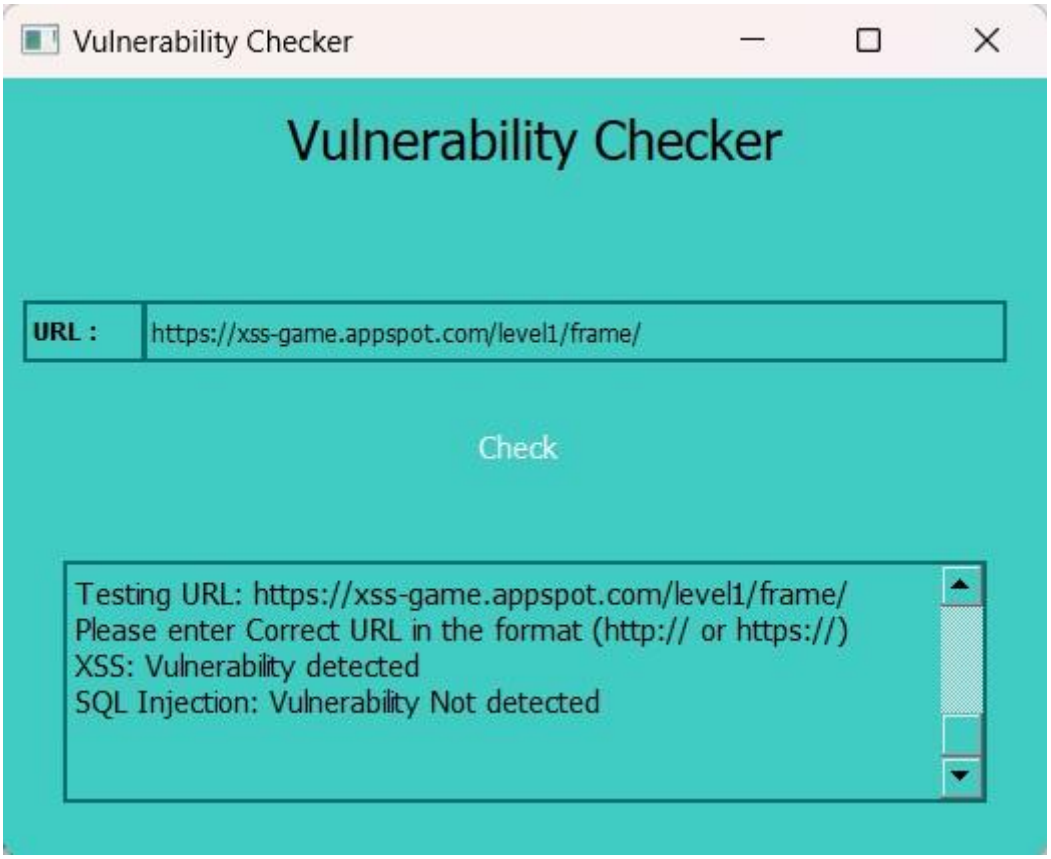

Fig 8.3: Output – 3

CHAPTER 9

CONCLUSION

With continuously evolving methods of hacking the detection of vulnerabilities becomes crucial. In this project we have created an GUI that can detect Phishing, Payload SQL injection and Reflected XSS vulnerability. The GUI can be used by anyone, even if they have no prior coding experience. It allows users to check any link just by copying it into the GUI and gives instant results. Random forest algorithms are used to detect phishing. Injecting SQL payloads and malicious scripts into the input fields of the given URL is the test of and detect the vulnerability of SQL Injections and Reflected XSS. When we get a particular output on load or script injection, the URL is classified as vulnerable to SQL injection or Reflected XSS. The GUI application can detect the given vulnerabilities in under 30 seconds with high accuracy and reliability.

CHAPTER 10

FUTURE WORK

- As known vulnerabilities are closely tracked by malicious players looking for their next big exploit, ,its crucial that software organizations work to stay one step ahead to the hackers, with a vulnerability management process that can detect and remediate the security vulnerabilities in their system.
- To provide frontend extension by the development of web applications for smartphones and extensions for various browsers. We can also work with other operating systems such as LINUX and MAC OS.

- Organizations that are able to put a program in place that allows them to continuosly track their development ecosystem, including its supply chain, providing automated prioritization, remediation, and reporting solutions, can look forward to a bright, secure future of smooth sprints and easy releases.

- To facilitate detection of various types of Open Web Application Security Project (OWASP) vulnerabilities such as click-jacking, Remote File Inclusion (RFI), Local File Inclusion (LFI) etc .

CHAPTER 11

REFERENCES

- Waleed Ali and Sharaf Malebary, ‘‘Particle Swarm Optimization-Based Feature Weighting for Improving Intelligent Phishing Website Detection,’’ https://ieeexplore.ieee.org/document/9121227
- Yazan Ahmad Alsariera; Victor Elijah Adeyemo; Abdullateef Oluwagbemiga Balogun ‘‘AI Meta-Learners and Extra-Trees Algorithm for the Detection of Phishing Websites,’’ https://ieeexplore.ieee.org/document/9154378
- Raima Banerjee and Aritra Baksi, ‘‘Detection of XSS attacks in a web application using Machine Learning Classifiers,’’ https://ieeexplore.ieee.org/document/9270052
- Arun Prasath Sivanesan, Akshay Mathur ‘‘A google chromium browser extension for detecting XSS attack in HTML5 based websites,’’ https://ieeexplore.ieee.org/document/8500284
- Mahdieh Zabihimayvan and Derek Doran, ‘‘Fuzzy rough set feature selection to enhance Phishing attack Detection,’’ https://ieeexplore.ieee.org/document/8858884
- Chen Ping, ‘‘A second order SQL injection detection method,’’ https://ieeexplore.ieee.org/document/8285104
- Xin Xie and Chunhui Ren, ‘‘ SQL Injection detection for web applications on Elastic-pooling CNN,” https://ieeexplore.ieee.org/document/8877739
- Gurvinder Kaur, ‘‘Study of XSS attacks and counter measures were proposed’’ http://ijcatr.com/archives/volume3/issue10/ijcatr0 3101001.pdf
- Mohammad A. Al-Jarrah, ‘‘Textual manipulation for SQL injection attacks,’’ https://www.researchgate.net/publication/2611816 34_Textual_Manipulation_for_SQL_Injection_At tacks
- Oluwatobi Ayodeji Akanbi, ‘‘General approaches of detecting the phishing attacks,’’ System using SVM and similarity Index, https://hcisjournal.springeropen.com/articles/10.1186/s13673 -017-0098-1